Question 5 - DevSecOps pipeline

Question 5: Integrate security into the CI/CD pipeline (DevSecOps): perform vulnerability/dependency scanning and demonstrate secure pipeline practices.

All commands below were run on the lab machine and their output is copied verbatim from the recorded logs in `evidence/logs/`. Repository: https://github.com/ksamirdev/ensp461-devops-lab3.

Pipeline

Jenkins runs in a container on the lab machine (`pipeline/Dockerfile`, `pipeline/compose.yml`). It polls the GitHub repository every 2 minutes. Builds 3, 4 and 5 were started by "an SCM change", with no manual trigger. Each scanner exits non-zero on a finding, which fails its stage and skips every later stage.

pipeline/Jenkinsfile

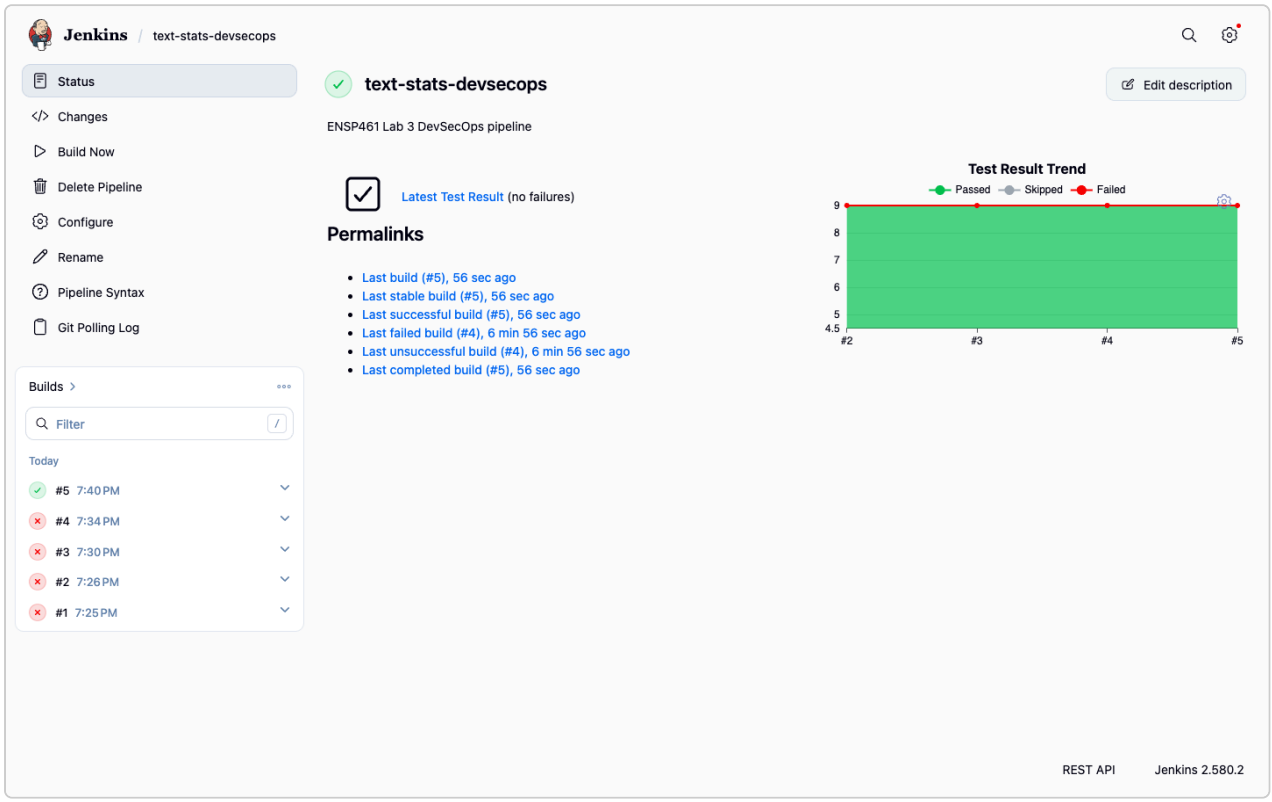
```
// DevSecOps gate: any stage failing blocks the build, so a vulnerable image never gets past it.
pipeline {
    agent any
    options { disableConcurrentBuilds() }
    triggers { pollSCM('H/2 * * * *') }

    stages {
        stage('Unit tests') {
            steps {
                sh 'python3 -m venv .venv'
                sh '.venv/bin/pip install -q -r requirements-dev.txt pip-audit bandit'
                sh '.venv/bin/python -m pytest -v --junitxml=test-results.xml'
            }
            post { always { junit 'test-results.xml' } }
        }
        stage('Dependency audit') {
            steps { sh '.venv/bin/pip-audit -r requirements-dev.txt' }
        }
        stage('Static analysis') {
            steps { sh '.venv/bin/bandit -r service -q' }
        }
        stage('Secrets and misconfiguration') {
            steps { sh 'trivy fs --scanners secret,misconfig --severity HIGH,CRITICAL --ignorefile .
trivyignore.yaml --exit-code 1 --skip-dirs .venv .' }
        }
        stage('Build image') {
            steps { sh 'docker build -t text-stats:ci-${BUILD_NUMBER} .' }
        }
        stage('Image scan') {
            steps { sh 'trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 text-sta
ts:ci-${BUILD_NUMBER}' }
        }
    }

    post {
        success { echo "Build ${BUILD_NUMBER} passed every security gate" }
        failure { echo "Build ${BUILD_NUMBER} BLOCKED by a security gate" }
    }
}
```

Build history: every gate caught something real

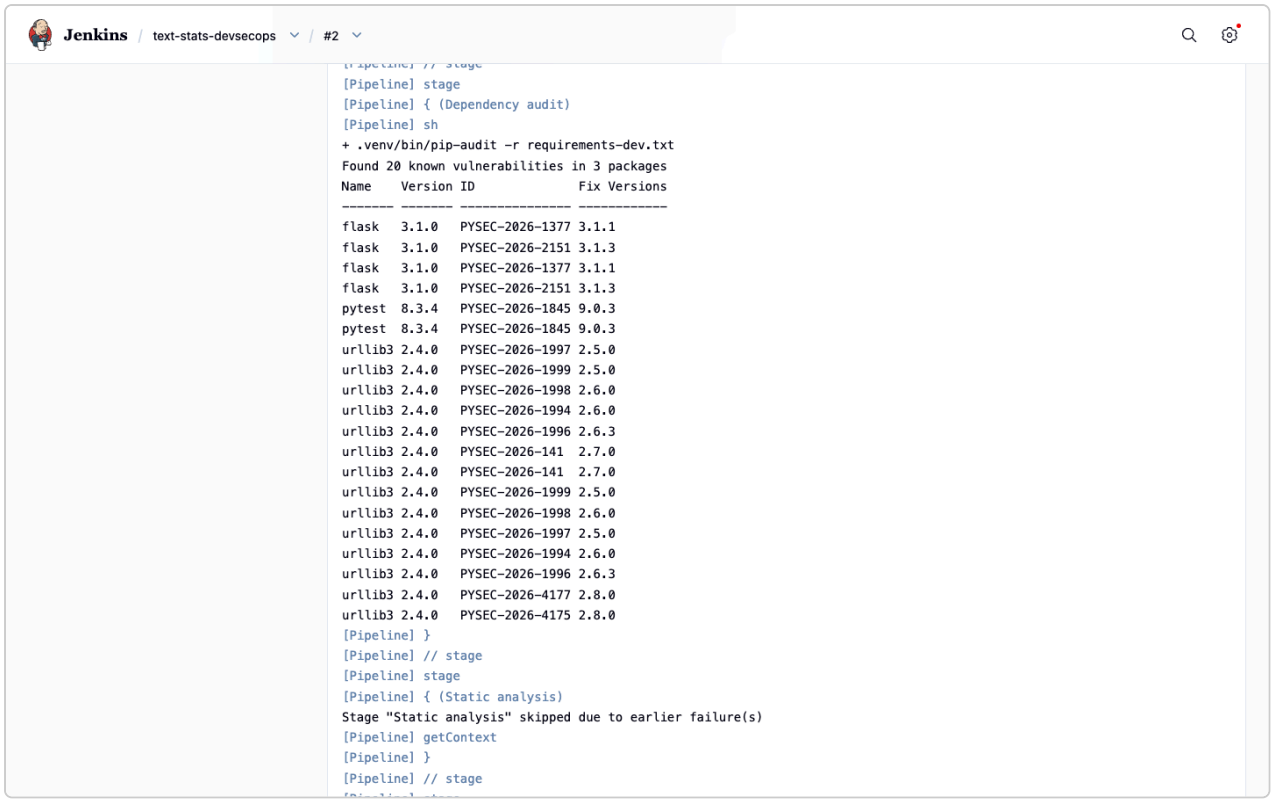
Build	Commit	Unit tests	Dependency audit	Bandit	Trivy fs	Trivy image	Result
#1	6a57381	pipeline/Jenkinsfile not pushed yet (setup error)					FAILURE
#2	2b3d7df	9 passed	**20 vulns** in flask, pytest, urllib3	skipped			BLOCKED
#3	9b358b9	9 passed	clean	clean	**4 HIGH** Dockerfile misconfigs	skipped	BLOCKED
#4	8a039d2	9 passed	clean	clean	clean	**5 HIGH** CVEs	BLOCKED
#5	7064c7e	9 passed	clean	clean	clean	clean	**SUCCESS**



Jenkins job: builds 1-4 failed, build 5 passed

Build 2: dependency audit (pip-audit)

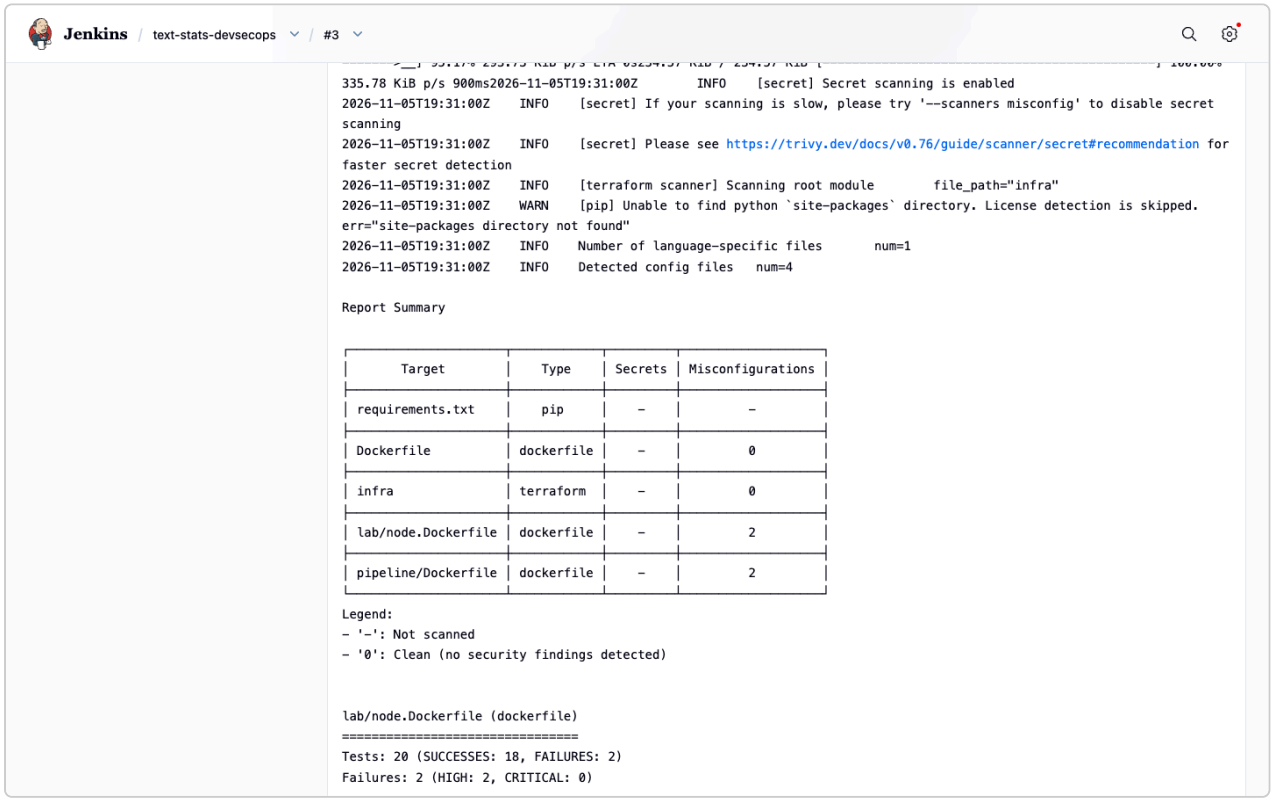
The requirements were copied from Lab 1, which pinned flask 3.1.0 and pytest 8.3.4. Both now have published advisories. urllib3 2.4.0 was pulled in by selenium 4.33. Fix: flask 3.1.3, pytest 9.2.0, and selenium 4.52 (which requires urllib3 >= 2.6.3).



Build 2 console: 20 known vulnerabilities in 3 packages, later stages skipped

Build 3: misconfiguration scan (Trivy fs)

The app Dockerfile passed, but both lab infrastructure images had 2 HIGH findings each. DS-0029 (missing `--no-install-recommends`) was fixed in both. DS-0002 (runs as root) is a real requirement for those two images: the Ansible node runs systemd as PID 1, and Jenkins needs the Docker socket. It is accepted in `.trivyignore.yaml`, scoped to those two files, each with a written reason. The application image gets no exceptions.



Build 3 console: Trivy misconfiguration findings

.trivyignore.yaml

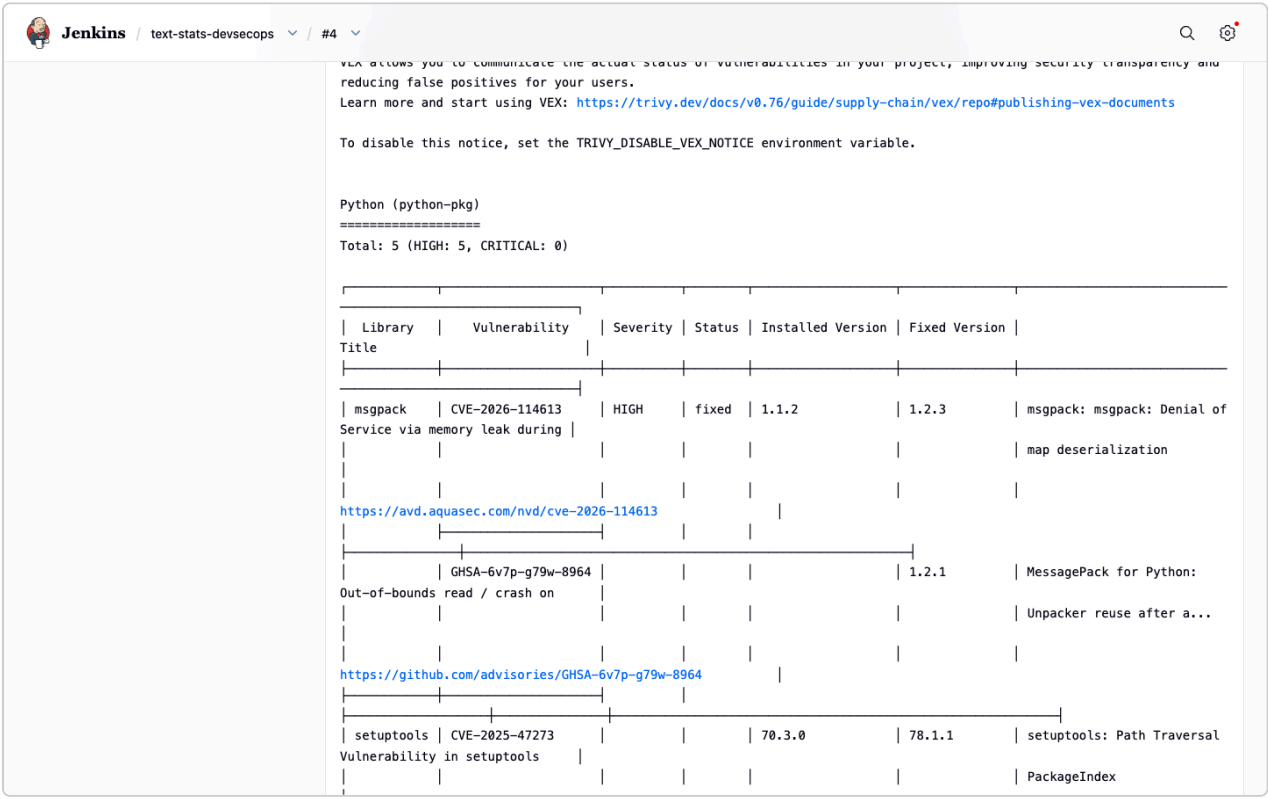
```
# Accepted risks, scoped to lab infrastructure only. The application image (Dockerfile) gets no exce
ptions.
misconfigurations:
  - id: DS-0002
    paths:
      - lab/node.Dockerfile
    statement: Ansible target node runs systemd as PID 1, which must be root. Lab-only, never deploy
ed.
  - id: DS-0002
    paths:
      - pipeline/Dockerfile
    statement: Jenkins controller needs root to use the host Docker socket. Bound to 127.0.0.1 on th
e lab machine.
```

Build 4: image scan (Trivy image)

The image had 5 HIGH CVEs in msgpack, urllib3 and setuptools. None of these are dependencies of the app: they are copies vendored inside pip in the Python base image. The app never runs pip, so the Dockerfile now uninstalls it after installing the requirements. This removes the CVEs and shrinks the attack surface.

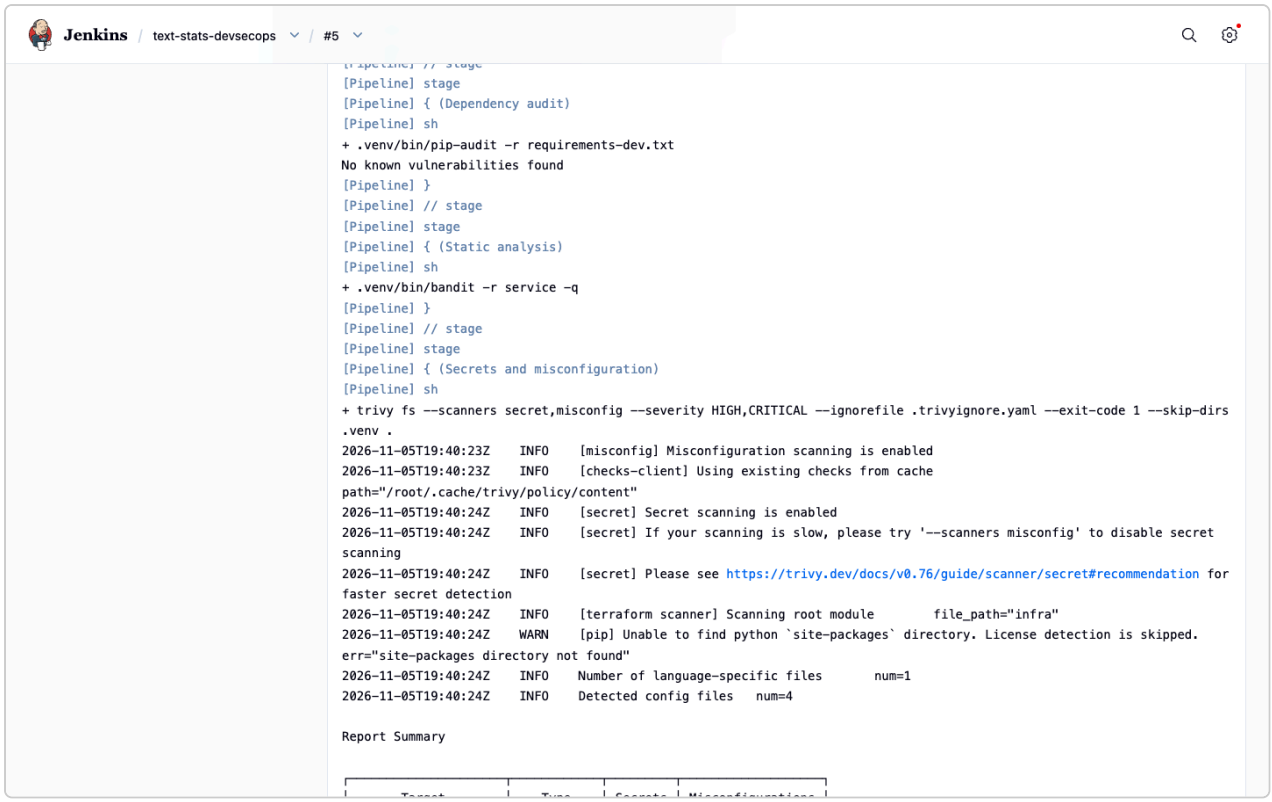
Package (vendored in pip)	Installed	Finding	Fixed in
msgpack	1.1.2	CVE-2026-114613 (DoS), GHSA-6v7p-g79w-8964 (out-of-bounds read)	1.2.3 / 1.2.1
setuptools (pkg_resources)	70.3.0	CVE-2025-47273 (path traversal)	78.1.1
urllib3	2.7.0	CVE-2026-97687, CVE-2026-97689	2.8.0

Summary of the local reproduction; full output in evidence/logs/q5-image-scan-before-fix.log

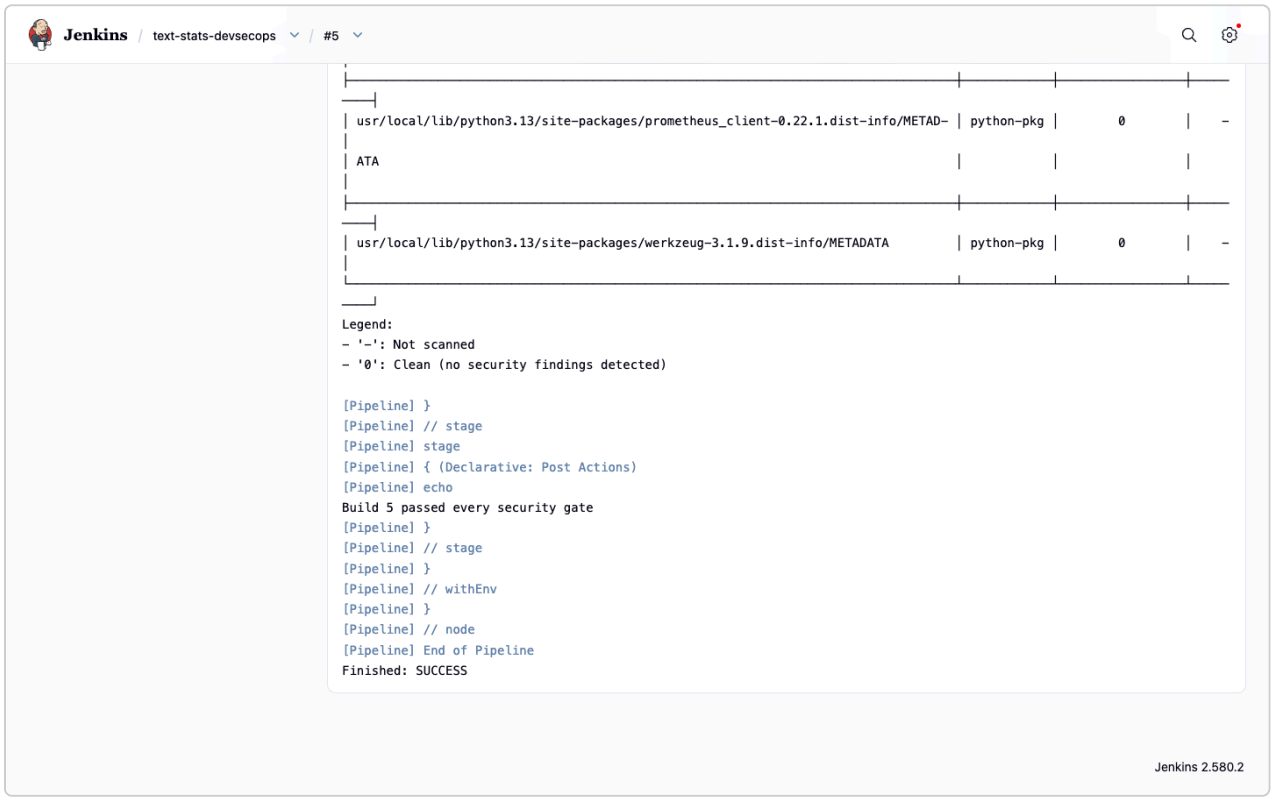


Build 4 console: image scan blocks on 5 HIGH CVEs

Build 5: all gates pass



Build 5: pip-audit and bandit clean, Trivy fs starting



Build 5: image scan clean, Finished: SUCCESS

Same scans run locally after the fixes

```
$ /home/samir/lab-tools/bin/pip-audit -r requirements-dev.txt
No known vulnerabilities found
$ /home/samir/lab-tools/bin/bandit -r service -q
```

```
$ docker build -q -t text-stats:scan .
sha256:c0e4e63db27052fd3f8a6e9959d585873ccacdc3ab05da6b496542a1bfeb244e
```

```
$ trivy fs --scanners secret,misconfig --severity HIGH,CRITICAL --ignorefile .trivyignore.yaml --exi
t-code 1 --skip-dirs .venv . 2>&1 | grep -v INFO
...
┌──────────────────────┬────────────┬─────────┬──────────────────┐
│ requirements.txt     │    pip     │    -    │         -        │
├──────────────────────┼────────────┼─────────┼──────────────────┤
│ Dockerfile           │ dockerfile │    -    │         0        │
├──────────────────────┼────────────┼─────────┼──────────────────┤
│ infra                │ terraform  │    -    │         0        │
├──────────────────────┼────────────┼─────────┼──────────────────┤
│ lab/node.Dockerfile  │ dockerfile │    -    │         0        │
├──────────────────────┼────────────┼─────────┼──────────────────┤
│ pipeline/Dockerfile  │ dockerfile │    -    │         0        │
└──────────────────────┴────────────┴─────────┴──────────────────┘
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
```

Bandit prints nothing with −q when it finds no issues, and its exit code is 0.

Secure pipeline practices used

- **Fail closed:** every scanner uses a non-zero exit code on findings, so nothing vulnerable reaches the image stage.
- **Shift left:** the cheap checks (tests, dependency audit, static analysis) run before the image is built.
- **Pinned versions:** every Python dependency and tool version (Trivy 0.76.2, Terraform 1.17.1) is pinned, so a build is reproducible and a scan result refers to known versions.
- **Accepted risk is explicit:** exceptions live in a reviewed file with a reason, not in a flag that silences everything.
- **Least privilege in the product:** the app image runs as an unprivileged user, has no pip, and binds only the port it needs.
- **Secret scanning:** Trivy's secret scanner runs on every build. The repository is public, so this matters.